



PYTHON

GHI CHÚ TOÀN TẬP

Trang 1

TRANG 1 — KIẾN THỨC NỀN TẢNG PYTHON

1 PYTHON LÀ GÌ?

Python là một ngôn ngữ lập trình bậc cao, thông dịch (interpreted), đa mục đích (general-purpose). Nó được tạo ra bởi **Guido van Rossum** và phát hành năm **1991**.

Python chú trọng đến khả năng đọc hiểu code (code readability) với cú pháp đơn giản, dễ hiểu.

2 CÁC ĐẶC ĐIỂM CỦA PYTHON

- ✓ Dễ học và dễ sử dụng
- ✓ Cú pháp rõ ràng, dễ đọc
- ✓ Là ngôn ngữ thông dịch (interpreted language)
- ✓ Là ngôn ngữ bậc cao (high-level language)
- ✓ Kiểu dữ liệu động (dynamically typed)
- ✓ Đa nền tảng (Windows, macOS, Linux)
- ✓ Thư viện chuẩn phong phú
- ✓ Hỗ trợ nhiều mô hình lập trình (Procedural, OOP, Functional)
- ✓ Mã nguồn mở và có cộng đồng lớn

3 CÀI ĐẶT & THIẾT LẬP PYTHON

- Tải Python từ python.org
- Chạy trình cài đặt và tick chọn "Add Python to PATH"
- Nhấn **Install Now**
- Kiểm tra bản cài đặt trong terminal/command prompt

```
$ python --version
Python 3.x.x
$ pip --version
pip x.x.x
```



4 BIẾN & HẲNG SỐ

- **Biến (Variables)** được dùng để lưu trữ dữ liệu.
- Trong Python, ta không cần khai báo kiểu dữ liệu tường minh.
- Python không có từ khóa riêng cho hằng số, nhưng theo quy ước ta dùng **UPPER CASE** cho hằng số.

```
name = "CodeWithZ" # string
age = 21             # integer
price = 19.99        # float
is_student = True    # boolean
PI = 3.14159         # hằng số (theo quy ước)
```

5 CÁC KIỂU DỮ LIỆU

int

Số nguyên
Ví dụ:
x = 10

float

Số thực
Ví dụ:
y = 3.14

str

Chuỗi ký tự
Ví dụ:
name = "Zaid"

bool

True hoặc False
Ví dụ:
is_ok = True

list

Có thứ tự, thay đổi được
Ví dụ:
nums = [1,2,3]

tuple

Có thứ tự, không đổi
Ví dụ:
t = (1,2,3)

set

Không thứ tự, duy nhất
Ví dụ:
s = {1,2,3}

dict

Cặp key-value
Ví dụ:
d = {'a':1, 'b':2}

★ **Lưu ý:** Python có kiểu dữ liệu động (dynamically typed), nên kiểu dữ liệu được xác định lúc chạy chương trình (runtime).

6 ÉP KIỂU DỮ LIỆU

- Chuyển đổi kiểu dữ liệu này sang kiểu khác.
- Dùng `int()`, `float()`, `str()`, `bool()`...

```
a = "10"
int_a = int(a) # "10" → 10
b = 5
float_b = float(b) # 5 → 5.0
c = 3.7
str_c = str(c) # 3.7 → "3.7"
d = 1
bool_d = bool(d) # 1 → True
```

7 NHẬP & XUẤT DỮ LIỆU

Nhập (Input):

```
name = input("Enter your name: ")
age = int(input("Enter age: "))
```

Xuất (Output):

```
print("Hello, " + name + "!")
print("You are", age, "years old.")
print(f"Next year you will be {age+1}")
```

8 TOÁN TỬ (TỔNG QUAN)

Số học: + - * / // % **
Gán: = += -= *= /= //= %=
So sánh: == != > < >= <=
Logic: and or not
Bitwise: & | ^ ~ << >>
Membership: in not in
Identity: is is not

9 CHÚ THÍCH (COMMENTS)

- Dùng comment để giải thích code của bạn.

10 CHƯƠNG TRÌNH PYTHON CƠ BẢN

```
name = "CodeWithZ"
age = 21
print(f"Xin chào, {name}!")
print(f"Bạn {age} tuổi.")
# Output:
# Xin chào, CodeWithZ!
# Bạn 21 tuổi.
```



PYTHON

GHI CHÚ TOÀN TẬP

Trang 2

TRANG 2 — ĐIỀU KHIỂN LƯỖNG (CONTROL FLOW)

Các câu lệnh điều khiển luồng (control flow) quyết định thứ tự thực thi các câu lệnh trong chương trình.

1 IF, ELIF, ELSE

- Dùng để đưa ra quyết định.
- Code trong điều kiện True đầu tiên sẽ được thực thi.
- Khối else là tùy chọn (optional).

```
age = 18
if age < 13:
    print("Child")
elif 13 <= age < 18:
    print("Teenager")
else:
    print("Adult")
```

age < 13?

Yes → No →

Child

2 IF LỒNG NHAU (NESTED IF)

- Một khối if hoặc else nằm bên trong if/else khác.

```
marks = 85
if marks >= 90:
    print("Grade A")
else:
    if marks >= 75:
        print("Grade B")
    else:
        print("Grade C")
```

💡 Mẹo: Thụt lề (indentation) rất quan trọng trong Python.

3 VÒNG LẶP FOR

- Dùng để lặp qua một chuỗi (list, string, tuple, range...).
- Thực thi khối lệnh cho mỗi phần tử.

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# Output:
apple
banana
cherry
```

a → b → c → ...

for item in iterable
do something

End

4 VÒNG LẶP WHILE

- Lặp lại một khối lệnh khi điều kiện còn True.
- Điều kiện được kiểm tra trước mỗi lần lặp.

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
# Output:
1 2 3 4 5
```

Start

i <= 5?

Yes → No → End

In i
i = i + 1

5 BREAK, CONTINUE, PASS

break → Dừng vòng lặp hoàn toàn.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
# Output: 1 2
```

continue → Bỏ qua lần lặp hiện tại và tiếp tục lần sau.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# Output: 1 2 4 5
```

pass → Không làm gì cả. Dừng như chỗ giữ (placeholder).

```
for i in range(3):
    if i == 1:
        pass # chờ code sau
    print(i)
# Output: 0 1 2
```

6 HÀM RANGE()

- Sinh ra một dãy số.
- range(start, stop, step)

```
range(5) → 0, 1, 2, 3, 4
range(1, 6) → 1, 2, 3, 4, 5
range(1, 10, 2) → 1, 3, 5, 7, 9
range(10, 0, -2) → 10, 8, 6, 4, 2
```

7 BIỂU THỨC ĐIỀU KIỆN

- Cách viết tắt của if-else.

```
# Cú pháp:
value_if_true if condition else value_if_false

age = 20
category = "Adult" if age >= 18 else "Minor"
print(category) # Output: Adult
```

8 VÍ DỤ THỰC HÀNH

Kiểm tra chẵn lẻ

```
n = int(input("Enter number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Tìm số lớn nhất trong 2 số

```
a = 10
b = 20
largest = a if a > b else b
print(largest)
```

Tổng n số tự nhiên đầu (while)

```
n = int(input("n: "))
i = 1
total = 0
while i <= n:
    total += i
    i += 1
print(total)
```

In bảng cửu chương

```
n = int(input("n: "))
for i in range(1, 11):
    print(n, "x", i, "=", n*i)
```



PYTHON

GHI CHÚ TOÀN TẬP

Trang 3

TRANG 3 — CHUỖI (STRINGS)

Chuỗi (string) trong Python là dãy ký tự được đặt trong dấu nháy đơn ('...'), nháy kép ("...") hoặc ba dấu nháy ('...' hoặc '...' hoặc '...').

1 TẠO CHUỖI

- Chuỗi là bất biến (immutable - không thể thay đổi).
- Có thể dùng nháy đơn, nháy kép hoặc ba nháy.

```
s1 = 'Hello' # nháy đơn
s2 = "Hello" # nháy kép
s3 = """Hello
World""" # ba nháy
```

Dùng ba dấu nháy cho chuỗi nhiều dòng. ★

2 INDEXING & SLICING

- Đánh chỉ số (indexing) bắt đầu từ 0.
- Chỉ số âm bắt đầu từ -1 (từ phải sang).

```
s = "Python"
Index: 0 1 2 3 4 5
Char : P y t h o n
Neg   : -6 -5 -4 -3 -2 -1
```

```
s[0] # 'P'
s[5] # 'n'
s[-1] # 'n'
s[1:4] # 'yth'
s[:3] # 'Pyt'
s[3:] # 'hon'
s[::-2] # 'Pto' (bước 2)
```

Ghi nhớ:
s[start:stop:step]
mặc định start = 0
mặc định stop = end
mặc định step = 1

3 CÁC PHƯƠNG THỨC CHUỖI (HAY DÙNG)

- `len(s)` → độ dài chuỗi
- `s.lower()` → chuyển về chữ thường
- `s.upper()` → chuyển về chữ hoa
- `s.strip()` → xóa khoảng trắng đầu & cuối
- `s.replace(old, new)` → thay thế chuỗi con
- `s.split(sep)` → tách chuỗi thành list
- `s.join(iterable)` → nối các phần tử bằng chuỗi
- `s.find(sub)` → vị trí đầu tiên của chuỗi con
- `s.count(sub)` → đếm số lần xuất hiện
- `s.startswith(sub)` → kiểm tra tiền tố
- `s.endswith(sub)` → kiểm tra hậu tố

★ Tất cả phương thức này trả về chuỗi mới (chuỗi gốc không đổi).

4 ĐỊNH DẠNG CHUỖI (STRING FORMATTING)

Dùng **f-strings** (Python 3.6+) - khuyến dùng

```
name = "Zaid"
age = 21
score = 95.5

msg = f"Hello {name}, you are {age} years old."
print(msg)
print(f"Score: {score:.1f}") # 95.5
```

Dùng **str.format()**

```
msg = "Hello {}, you are {} years old.".format(name, age)
print(msg)
```

Dùng định dạng **%** (kiểu cũ)

```
msg = "Hello %, you are %d years old." % (name, age)
print(msg)
```

5 KÝ TỰ ĐẶC BIỆT (ESCAPE CHARACTERS)

- `\n` → Xuống dòng
- `\t` → Tab
- `\r` → Carriage return
- `\'` → Nháy đơn
- `\"` → Nháy kép
- `\\` → Dấu gạch chéo ngược

Ví dụ:
s = "Hello\nWorld\t"
😊
print(s)
Output:
Hello
World 😊

6 TÍNH BẤT BIẾN CỦA CHUỖI

- Sau khi tạo, chuỗi không thể thay đổi.
- Mọi chỉnh sửa đều tạo ra chuỗi mới.

```
s = "Python"
s[0] = 'p' ❌ Lỗi!
s = 'P' + s[1:] ✅ Đúng cách
print(s) # Python
```

Chuỗi bất biến (immutable), nhưng ta có thể tạo chuỗi mới. ★

7 VÍ DỤ THỰC HÀNH

Ví dụ 1: Đảo ngược chuỗi

```
s = "hello"
rev = s[::-1]
print(rev) # olleh
(Step -1 nghĩa là đảo ngược)
```

Ví dụ 2: Kiểm tra chuỗi đối xứng

```
s = input("Enter text: ")
clean = s.replace(" ", "").lower()
print(clean == clean[::-1])
# True nếu là palindrome
```

Ví dụ 3: Đếm nguyên âm

```
s = "Hello World".lower()
vowels = 'aeiou'
count = sum(1 for ch in s if ch in vowels)
print(count) # 3
```

Ví dụ 4: Xóa khoảng trắng

```
s = "Hello Python World"
new_s = s.replace(" ", "")
print(new_s) # HelloPythonWorld
```



PYTHON

GHI CHÚ TOÀN TẬP

Trang 4

TRANG 4 — LIST, TUPLE & SET

Python có sẵn nhiều cấu trúc dữ liệu dùng để lưu trữ tập hợp dữ liệu.

1 LIST

- Tập hợp có thứ tự (ordered).
- Có thể thay đổi (mutable).
- Cho phép giá trị trùng lặp.
- Phần tử có thể là bất kỳ kiểu dữ liệu nào.

```
nums = [1, 2, 3, 4, 5]
mixed = ["hello", 10, 3.5, True]
nested = [1, [2, 3], {"a": 1}]
```

INDEXING & SLICING

Đánh chỉ số bắt đầu từ 0.

```
a = ['p','y','t','h','o','n']
index: 0 1 2 3 4 5
-index: -6 -5 -4 -3 -2 -1

a[0] → 'p'
a[3] → 'h'
a[-1] → 'n'
a[1:4] → ['y','t','h']
```

2 TUPLE

- Tập hợp có thứ tự.
- Không thể thay đổi (immutable).
- Cho phép giá trị trùng lặp.
- Nhanh hơn list.

```
t = (10, 20, 30, 40)
mixed = ('a', 2, 3.5, True)
single = (5,) # cần dấu phẩy
```

GIẢI NÉN TUPLE (UNPACKING)

Gán các phần tử tuple cho biến.

```
point = (10, 20)
x, y = point
print(x, y) # 10 20

a, b, *rest = (1, 2, 3, 4, 5)
print(a, b, rest) # 1 2 [3, 4, 5]
```

4 LIST COMPREHENSION

- Cách tạo list ngắn gọn.

```
[expression for item in iterable if condition]
```

VÍ DỤ

```
squares = [x**2 for x in range(6)]
# [0, 1, 4, 9, 16, 25]

even = [x for x in range(1, 11) if x % 2 == 0]
# [2, 4, 6, 8, 10]

words = [w.upper() for w in ['hi', 'python', 'code']]
# ['HI', 'PYTHON', 'CODE']

matrix = [[i*j for j in range(3)] for i in range(3)]
# [[0,0,0],[0,1,2],[0,2,4]]
```

CÁC PHƯƠNG THỨC THÔNG DỤNG

append(x) → thêm vào cuối
insert(i, x) → thêm vào vị trí i
extend(iterable) → thêm nhiều phần tử
remove(x) → xóa lần xuất hiện đầu tiên
pop([i]) → xóa & trả về phần tử cuối hoặc tại i
clear() → xóa hết phần tử
index(x) → trả về vị trí xuất hiện đầu tiên
count(x) → đếm số lần xuất hiện
sort() → sắp xếp tăng dần
reverse() → đảo ngược list

3 SET

- Tập hợp không có thứ tự, phần tử duy nhất.
- Có thể thay đổi (mutable).
- Không có giá trị trùng lặp.
- Hữu ích khi kiểm tra thành viên & phép toán tập hợp.

```
s = {1, 2, 3, 4, 4} # số 4 trùng sẽ bị bỏ
s = set([1, 2, 3]) # set() constructor
```

CÁC PHÉP TOÁN TẬP HỢP

```
a = {1, 2, 3}
b = {3, 4, 5}
a | b → {1,2,3,4,5} # hợp
a & b → {3} # giao
a - b → {1, 2} # hiệu
a ^ b → {1,2,4,5} # đối xứng
3 in a → True # thành viên
```

5 BẢNG SO SÁNH

Đặc điểm	List	Tuple	Set
Có thứ tự	Yes	Yes	No
Thay đổi được	Yes	No	Yes

KHI NÀO DÙNG CÁI GÌ?

- Dùng **List** khi cần tập hợp có thứ tự và có thể thay đổi.
- Dùng **Tuple** khi cần tập hợp có thứ tự nhưng cố định.

GHI CHÚ NHANH

- List có tính linh động (dynamic).
- Tuple an toàn hơn cho dữ liệu cố định (constant data).



PYTHON

GHI CHÚ TOÀN TẬP

Trang 5

TRANG 5 — DICTIONARY & COMPREHENSION

Dictionary lưu trữ dữ liệu theo cặp key-value. Comprehension cung cấp cách ngắn gọn để tạo list, set và dictionary.

1 DICTIONARY CƠ BẢN

- Dictionary là tập hợp các cặp key-value.
- Key phải duy nhất và bất biến (str, int, tuple...).
- Value có thể là bất kỳ kiểu nào và được phép trùng.

```
student = {  
    "name": "Zaid",  
    "age": 21,  
    "course": "B.Tech",  
    "is_hosteller": True  
}
```

Key → Value

"name" → "Zaid"

"age" → 21

"course" → "B.Tech"

2 TRUY CẬP & CHỈNH SỬA

- Truy cập bằng key.
- Thêm hoặc cập nhật cặp key-value.
- remove() / pop() / del để xóa phần tử.

```
student["name"] # Zaid  
student.get("age") # 21  
student["city"] = "Delhi" # thêm mới  
student["age"] = 22 # cập nhật  
student.pop("course") # xóa & trả về  
del student["is_hosteller"] # xóa
```

★ get() an toàn hơn [] vì nó trả về None nếu không tìm thấy key.

3 CÁC PHƯƠNG THỨC DICTIONARY

- keys()** → trả về mọi key
- values()** → trả về mọi value
- items()** → trả về cặp key-value
- get(key)** → trả về value của key
- pop(key)** → xóa key & trả về value
- update(dict)** → cập nhật từ dict khác
- clear()** → xóa hết phần tử
- copy()** → trả về bản sao (shallow)
- setdefault()** → trả về value của key, nếu chưa có thì thêm với giá trị mặc định

★ Dictionary không có thứ tự (Python < 3.7). Từ Python 3.7+, dict giữ đúng thứ tự chèn (insertion order).

4 NESTED DICTIONARY

- Dictionary lồng bên trong dictionary khác.

```
students = {  
    1: {"name": "Zaid", "age": 21},  
    2: {"name": "Ali", "age": 20},  
    3: {"name": "Sara", "age": 22}  
}  
print(students[2]["name"]) # Ali
```

1 → {"name": "Zaid", "age": 21}

2 → {"name": "Ali", "age": 20}

3 → {"name": "Sara", "age": 22}

5 DICTIONARY COMPREHENSION

Tạo dictionary chỉ trong một dòng.

```
squares = {x: x*x for x in range(1,6)}  
# {1:1, 2:4, 3:9, 4:16, 5:25}  
  
upper = {k: v.upper() for k,v in student.items()}
```

6 LIST COMPREHENSION

Tạo list ngắn gọn.

```
squares = [x*x for x in range(1,6)]  
# [1, 4, 9, 16, 25]  
  
even = [x for x in range(1,11) if x%2==0]
```

7 SET COMPREHENSION

Tạo set ngắn gọn.

```
squares = {x*x for x in range(1,6)}  
unique_chars = {ch for ch in "hello world"}  
vowels = {ch for ch in "hello" if ch in "aeiou"}
```

8 NESTED COMPREHENSION

Hữu ích cho list 2D hoặc bài toán phức tạp.

```
matrix = [[i*j for j in range(1,4)]  
           for i in range(1,4)]  
# [[1,2,3], [2,4,6], [3,6,9]]  
  
coords = [(x,y) for x in range(2) for y in range(2)]  
# [(0,0), (0,1), (1,0), (1,1)]
```

9 VÍ DỤ THỰC HÀNH

Đếm tần suất phần tử

```
nums = [1,2,2,3,3,3,4]  
freq = {}  
for n in nums:  
    freq[n] = freq.get(n,0)+1
```

Hoán đổi key/value

```
d = {'a':1, 'b':2, 'c':3}  
rev = {v:k for k,v in d.items()}
```

Lọc dictionary

```
scores = {'a':45, 'b':89, 'c':60, 'd':90}  
passed = {k:v for k,v in scores.items() if v>=60}
```

Gộp hai dictionary

```
d1 = {'a':1, 'b':2}  
d2 = {'b':20, 'c':30}  
merged = {**d1, **d2} # Python 3.5+
```



TRANG 6 — HÀM (FUNCTIONS)

Hàm (functions) giúp tổ chức code thành các khối có thể tái sử dụng. Python có nhiều cách để định nghĩa hàm và truyền các loại đối số khác nhau.

1 ĐỊNH NGHĨA HÀM

- Dùng từ khóa **def**.
- Code trong hàm chỉ chạy khi được gọi.
- Thụt lề (indentation) rất quan trọng.

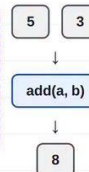
```
def greet():  
    print("Hello, Welcome to Python!")  
  
greet() # gọi hàm
```

Hàm giúp code DRY
(Don't Repeat Yourself).

2 HÀM CÓ THAM SỐ

- Ta có thể truyền dữ liệu vào hàm bằng tham số.
- Tham số đóng vai trò như chỗ giữ (placeholder).

```
def add(a, b):  
    sum = a + b  
    return sum  
  
result = add(5, 3)  
print("Sum =", result) # Sum = 8
```



3 CÂU LỆNH RETURN

- return** gửi kết quả về nơi gọi hàm.
- Khi return được thực thi, hàm kết thúc.

```
def square(x):  
    return x * x  
  
n = square(4)  
print(n) # 16
```

Nếu không có return,
hàm mặc định trả về None.

4 ĐỐI SỐ MẶC ĐỊNH

- Cung cấp giá trị mặc định cho tham số.
- Nếu không truyền, giá trị mặc định sẽ được dùng.

```
def greet(name, msg="Hello!"): # Hello! Zaid  
    print(msg, name)  
  
greet("Zaid") # Welcome Zaid  
greet("Zaid", "Welcome") # Welcome Zaid
```

Đối số mặc định giúp
hàm linh hoạt hơn. ★

5 ĐỐI SỐ THEO TỪ KHÓA

- Chỉ định tên tham số khi gọi hàm.
- Thứ tự không quan trọng.

```
def info(name, age, city):  
    print(name, age, city)  
  
info(city="Delhi", age=21, name="Zaid")  
# Zaid 21 Delhi
```

Dùng keyword
arguments để code dễ
đọc hơn. 😊

6 *ARGS: ĐỐI SỐ ĐỘ DÀI BIẾN ĐỔI

- *args** cho phép hàm nhận số lượng bất kỳ đối số vị trí.

```
def total(*numbers):  
    s = 0  
    for n in numbers:  
        s += n  
    return s  
  
print(total(1, 2, 3)) # 6  
print(total(5, 10, 15, 20)) # 50
```

*args được đóng gói
thành tuple bên trong
hàm. type(numbers) là
tuple.

7 **KWARGS: ĐỐI SỐ TỪ KHÓA ĐỘ DÀI BIẾN ĐỔI

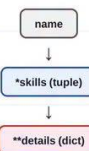
- **kwargs** cho phép hàm nhận số lượng bất kỳ đối số từ khóa.

```
def show(**details):  
    for key, value in details.items():  
        print(key, "=", value)  
  
show(name="Zaid", age=21, city="Delhi")  
# name = Zaid  
# age = 21  
# city = Delhi
```

**kwargs được đóng gói
thành dictionary bên
trong hàm.

8 *ARGS VÀ **KWARGS CÙNG NHAU

```
def profile(name, *skills, **details):  
    print("Name:", name)  
    print("Skills:", skills)  
    for k, v in details.items():  
        print(f" {k} = {v}")  
  
profile("Zaid", "Python", "Java", "SQL", age=21, city="Delhi")
```



9 HÀM LAMBDA

- Hàm ẩn danh (anonymous) nhỏ gọn.
- Hữu ích cho các thao tác đơn giản, ngắn.

```
square = lambda x: x * x  
print(square(5)) # 25  
  
add = lambda a, b: a + b  
print(add(10, 20)) # 30
```

Cú pháp:

lambda arguments:
expression

Dùng với map(), filter(),
v.v. ★

10 ĐỆ QUY (HÀM GỌI CHÍNH NÓ)

- Hữu ích cho bài toán có thể chia thành bài toán con tương tự.
- Phải có điều kiện dừng (base condition).

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5)) # 120
```

Call Stack (n=3)



11 PHẠM VI BIẾN (SCOPE)

```
x = 10 # biến toàn cục (global)
```



PYTHON

GHI CHÚ TOÀN TẬP

Trang 7

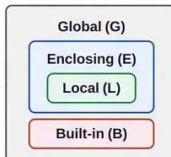
TRANG 7 — SCOPE, HÀM DỰNG SẴN & THỰC HÀNH

Hiểu về scope giúp ta tránh lỗi. Các hàm dựng sẵn (built-in functions) giúp code ngắn gọn, sạch và hiệu quả hơn.

1 SCOPE TRONG PYTHON

- Scope xác định phạm vi hiển thị (visibility) của biến.
- Python tuân theo quy tắc **LEGB**.

L - Local: bên trong hàm
E - Enclosing: trong hàm bao ngoài
G - Global: bên ngoài mọi hàm
B - Built-in: tên dựng sẵn của Python



```
x = 10 # Global
def outer():
    y = 20 # Enclosing
    def inner():
        z = 30 # Local
        print(x, y, z)
    inner()
outer() # Output: 10 20 30
```

2 TỪ KHÓA GLOBAL & NONLOCAL

- global** - dùng để thay đổi biến toàn cục từ bên trong hàm.
- nonlocal** - dùng để thay đổi biến trong hàm bao ngoài.

ví dụ global

```
x = 5
def change():
    global x
    x = 10
change()
print(x) # 10
```

ví dụ nonlocal

```
def outer():
    x = 5
    def inner():
        nonlocal x
        x = 10
    inner()
    print(x) # 10
outer()
```

⚡ Mẹo: Dùng global cẩn thận. Quá nhiều biến global khiến code khó bảo trì!

3 CÁC HÀM DỰNG SẴN QUAN TRỌNG

Hàm	Mô tả	Ví dụ	Kết quả
abs(x)	Giá trị tuyệt đối	abs(-7)	7
len(s)	Độ dài đối tượng	len("Python")	6
type(x)	Kiểu dữ liệu	type(10.5)	<class 'float'>
max(iterable)	Giá trị lớn nhất	max(1,5,2)	5
min(iterable)	Giá trị nhỏ nhất	min(1,5,2)	1
sum(iterable)	Tổng các giá trị	sum([1,2,3])	6
round(x, n)	Làm tròn n chữ số	round(3.1415,2)	3.14

Hàm	Mô tả	Ví dụ	Kết quả
sorted(x)	Trả về list đã sắp xếp	sorted([3,1,2])	[1,2,3]
reverse(x)	Đảo ngược một dãy	list.reverse([1,2,3])	[3,2,1]
enumerate(x)	Đánh chỉ số kèm giá trị	list(enumerate(['a','b']))	[(0,'a'), (1,'b')]
zip(a, b)	Kết hợp các dãy	list(zip([1,2],[3,4]))	[(1,3), (2,4)]
range(a,b,c)	Dãy số	list(range(1,6,2))	[1,3,5]
input()	Nhận dữ liệu từ người dùng	input("Name: ")	User input
print()	Hiển thị ra màn hình	print("Hello")	Hello

4 VÍ DỤ THỰC HÀNH

Đếm số từ trong câu

```
sentence = "Python is fun and powerful"
words = sentence.split()
print(len(words)) # 5
```

Tìm số lớn nhất

```
nums = [12, 45, 7, 89, 34]
largest = max(nums)
print(largest) # 89
```

Kiểm tra chuỗi đối xứng

```
s = input("Enter text: ")
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Giai thừa dùng hàm

```
def fact(n):
    if n == 0 or n == 1:
        return 1
    return n * fact(n-1)
print(fact(5)) # 120
```

Tổng các chữ số

```
n = 12345
total = 0
while n > 0:
    total += n % 10
    n //= 10
print(total) # 15
```

Chẵn hay lẻ

```
n = int(input("Enter number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

5 TỰ THỰC HÀNH THÊM (SHORT PRACTICE)

- ① Viết hàm in ra một dãy chẵn/lẻ ② In tất cả số nguyên tố ③ Lấy list các số ④ Hoán đổi hai số ⑤ In bảng cửu chương



PYTHON

GHI CHÚ TOÀN TẬP

Trang 8

TRANG 8 — NỀN TẢNG OOP

Lập trình hướng đối tượng (OOP) giúp ta mô hình hóa các thực thể trong thế giới thực bằng Object và Class.

1 OOP LÀ GÌ?

- OOP là mô hình lập trình dựa trên khái niệm "Object" chứa dữ liệu (attributes) và code (methods).
- Các trụ cột chính của OOP: **Encapsulation, Abstraction, Inheritance, Polymorphism**.

VÍ DỤ THỰC TẾ

Class → Bản thiết kế (Car)
Object → Thực thể (MyCar)
Attributes → color, speed
Methods → start(), stop()

2 CLASS VÀ OBJECT

- Class là bản thiết kế (blueprint) hoặc khuôn mẫu.
- Object là một thực thể (instance) của class.
- Ta có thể tạo nhiều object từ một class.

```
class Car:          # Class
    pass

c1 = Car()          # Object 1
c2 = Car()          # Object 2
```

Think: Class = Bản thiết kế
Object = Sản phẩm thực tế ★

3 HÀM KHỞI TẠO __init__()

- __init__()** là phương thức đặc biệt (constructor).
- Được gọi tự động khi object được tạo.
- Dùng để khởi tạo các thuộc tính (attributes).

```
class Student:
    def __init__(self, name, age):
        self.name = name # constructor
        self.age = age

s1 = Student("Zaid", 21)
print(s1.name) # Zaid
print(s1.age)  # 21
```

4 BIẾN INSTANCE

- Biến khai báo trong **__init__()** bằng self.
- Mỗi object có bản sao riêng.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Alice", 20)
p2 = Person("Bob", 25)
print(p1.name, p1.age) # Alice 20
print(p2.name, p2.age) # Bob 25
```

Memory View

p1
name: "Alice"
age: 20

p2
name: "Bob"
age: 25

5 BIẾN CLASS (CLASS VARIABLES)

- Khai báo trong class nhưng ngoài mọi method.
- Được chia sẻ bởi tất cả các object.

```
class Counter:
    count = 0 # biến class
    def __init__(self):
        Counter.count += 1

c1 = Counter()
c2 = Counter()
print(Counter.count) # 2
```

Thay đổi ở biến class ảnh hưởng tới mọi object.
Truy cập bằng `ClassName.var` hoặc `self.var` ★

6 INSTANCE METHODS

- Hàm định nghĩa trong class dùng self làm tham số đầu.
- Thao tác trên dữ liệu của object (instance).

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def start(self):
        return f"{self.brand} started"

mycar = Car("Toyota")
print(mycar.start()) # Toyota started
```

Method Call

mycar.start()
↓
start(self)

7 TỬ KHÓA SELF

- self tham chiếu đến object hiện tại.
- Giúp truy cập biến và phương thức của instance.

Không có self ❌

```
class Test:
    def __init__(name):
        name = "Zaid"
t = Test()
# Lỗi: thiếu 1 đối số
```

Có self ✅

```
class Test:
    def __init__(self):
        self.name = "Zaid"
t = Test()
print(t.name) # Zaid
```

Luôn dùng self để gắn biến với object! ★

8 SƠ ĐỒ OBJECT

Class: Student
Attributes: name, age
Methods: __init__(), display()



Object: s1
name: "Zaid"
age: 21

Object: s2
name: "Sara"
age: 20

Mỗi object có dữ liệu (state) riêng nhưng dùng chung phương thức của class. ★

9 VÍ DỤ THỰC HÀNH

1) Bank Account

```
class Account:
    def __init__(self, bal):
        self.balance = bal
    def deposit(self, amt):
        self.balance += amt
```

2) Rectangle Area

```
class Rectangle:
    def __init__(self, l, b):
        self.l = l
        self.b = b
```

3) Employee

```
class Employee:
    def __init__(self, name, sal):
        self.name = name
        self.sal = sal
```



TRANG 9 — OOP NÂNG CAO

Các khái niệm này giúp chương trình mạnh mẽ hơn, linh hoạt hơn và sẵn sàng cho thực tế.

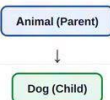
1 TÍNH KẾ THỪA (INHERITANCE)

- Kế thừa cho phép một class (con) thừa hưởng thuộc tính và phương thức từ class khác (cha).
- Thúc đẩy tái sử dụng code và thể hiện quan hệ trong thực tế.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal): # Dog kế thừa Animal
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak() # Animal speaks
d.bark() # Dog barks
```



2 ĐA KẾ THỪA (MULTIPLE INHERITANCE)

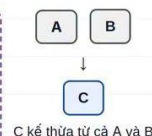
- Một class có thể kế thừa từ nhiều class cha.
- Python hỗ trợ đa kế thừa.

```
class A:
    def methodA(self):
        print("Method of A")

class B:
    def methodB(self):
        print("Method of B")

class C(A, B): # C kế thừa A và B
    pass

obj = C()
obj.methodA() # Method of A
obj.methodB() # Method of B
```



Dùng đa kế thừa cẩn thận để tránh phức tạp. ⭐

3 GHI ĐỀ PHƯƠNG THỨC (METHOD OVERRIDING)

- Class con cung cấp cách hiện thực riêng cho phương thức đã có ở class cha.
- Thực hiện bằng cách dùng cùng tên phương thức.

```
class Parent:
    def show(self):
        print("This is Parent")

class Child(Parent):
    def show(self):
        print("This is Child")

c = Child()
c.show() # This is Child
```

💡 Tại sao? Cho phép class con có hành vi riêng trong khi vẫn dùng class cha. ⭐

4 ĐA HÌNH (POLYMORPHISM)

- Khả năng của object thể hiện ở nhiều dạng khác nhau.
- Cùng tên phương thức, hành vi khác nhau.
- 2 loại: (1) Method Overriding (2) Method Overloading (dùng default args / *args / **kwargs)

```
class Shape:
    def area(self): pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r * self.r

class Rectangle(Shape):
    def __init__(self, l, b): self.l, self.b = l, b
    def area(self): return self.l * self.b

shapes = [Circle(5), Rectangle(4, 3)]
for s in shapes:
    print(s.area()) # Đa hình thể hiện ở đây
```

5 TÍNH ĐÓNG GÓI (ENCAPSULATION)

- Gói dữ liệu (biến) và phương thức lại trong một đơn vị (class).
- Hạn chế truy cập trực tiếp bằng access modifiers.

```
class Bank:
    def __init__(self, bal):
        self._balance = bal #
    def deposit(self, amt):
        self._balance += amt
    def get_balance(self):
        return self._balance
```

Access Modifiers

__variable → protected
_variable → private (name mangling)
Không có tiền tố → public

💡 Biến private không thể truy cập trực tiếp từ ngoài class.

6 TÍNH TRỪU TƯỢNG (ABSTRACTION)

- Ẩn chi tiết cài đặt bên trong, chỉ hiển thị tính năng cần thiết.
- Đạt được bằng Abstract Classes và Interfaces.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Triangle(Shape):
    def area(self):
        return "Area of Triangle"
```

Abstract class không thể khởi tạo trực tiếp. Class con phải hiện thực các abstract method. ⭐

7 HÀM SUPER()

- Dùng để gọi phương thức của class cha.
- Hữu ích khi class con ghi đè phương thức nhưng vẫn cần chức năng của cha.

```
class Parent:
    def __init__(self):
        print("Parent Constructor")
    def display(self):
        print("Parent Method")

class Child(Parent):
    def __init__(self):
        super().__init__() # gọi constructor cha
        print("Child Constructor")
    def display(self):
        super().display() # gọi method cha
        print("Child Method")
```

super() giúp viết code sạch, dễ bảo trì. ⭐

8 PHƯƠNG THỨC MA THUẬT (MAGIC/DUNDER)

- Phương thức đặc biệt có 2 gạch dưới ở đầu và cuối.
- Cung cấp nạp chồng toán tử (operator overloading) và hành vi đặc biệt.

__init__() → Constructor
__str__() → Biểu diễn dạng chuỗi
__len__() → Độ dài object
__add__() → toán tử +
__eq__() → toán tử ==

Ví dụ: __str__()

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return f'Book: {self.title}'
```



PYTHON

GHI CHÚ TOÀN TẬP

Trang 10

TRANG 10 — FILE, NGOẠI LỆ & BEST PRACTICES

Các khái niệm này giúp viết chương trình mạnh mẽ, xử lý lỗi khéo léo và tuân theo các nguyên tắc tốt để có code sạch, hiệu quả.

1 LÀM VIỆC VỚI FILE

- File dùng để lưu trữ dữ liệu lâu dài.
- Python có sẵn hàm để làm việc với file.

Mở file: `open(file, mode)`

Mode	Mô tả
r	Đọc (mặc định)
w	Ghi (ghi đè file)
a	Nối thêm (thêm vào cuối)
x	Tạo mới (lỗi nếu đã tồn tại)
r+	Đọc và ghi
w+	Ghi và đọc
a+	Nối thêm và đọc

```
# Ghi vào file
f = open("data.txt", "w")
f.write("Hello World!\n")
f.write("Python is awesome.")
f.close()

# Đọc từ file
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

2 CÂU LỆNH WITH (BEST PRACTICE)

- Tự động đóng file.
- Cách tốt hơn để làm việc với file.

```
with open("data.txt", "a") as f:
    f.write("\nLearning Python Daily!")

with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Ưu điểm: File sẽ tự đóng ngay cả khi có lỗi xảy ra. ★

Các phương thức file thông dụng

`read(size)` → đọc theo kích thước
`readline()` → đọc một dòng
`readlines()` → đọc mọi dòng vào list
`write(str)` → ghi một chuỗi
`writelines(list)` → ghi list các chuỗi

3 XỬ LÝ NGOẠI LỆ

- Exception là lỗi xảy ra khi chạy chương trình.
- Ta xử lý chúng để tránh crash chương trình.

```
try:
    x = int(input("Enter a number:"))
    result = 10 / x
    print("Result:", result)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input! Enter a number.")
else:
    print("Everything went fine!")
finally:
    print("This runs no matter what.")
```

Lỗi thường gặp

ZeroDivisionError - chia cho 0
ValueError - giá trị không hợp lệ
TypeError - sai kiểu dữ liệu
FileNotFoundError - không thấy file
IndexError - chỉ số không hợp lệ

4 TỰ NÉM NGOẠI LỆ (RAISING)

- Ta có thể tự ném exception bằng từ khóa `raise`.
- Giúp xử lý lỗi cụ thể.

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be 18 or above")
    return "Access Granted"

try:
    age = int(input("Enter your age: "))
    print(check_age(age))
except ValueError as e:
    print("Error:", e)
```

Chỉ `raise` exception có ý nghĩa, với thông báo rõ ràng.

5 CÁC NGUYÊN TẮC TỐT (BEST PRACTICES)

1. Tuân theo PEP 8

Tuân theo phong cách viết code chuẩn của Python (thụt lề, đặt tên, khoảng cách...).

2. Đặt tên rõ nghĩa

Dùng tên biến, tên hàm rõ ràng và mô tả đúng ý nghĩa.
Sai: `x = 10`
Đúng: `age = 10`

3. Giữ hàm nhỏ gọn

Hàm nhỏ để đọc, dễ test và dễ bảo trì hơn.

4. Tránh số ma thuật (magic numbers)

Dùng hằng số hoặc biến đặt tên thay vì số cứng.
Sai: `if age >= 18:`
Đúng: `ADULT_AGE=18`

5. Viết comment cho code

Viết comment ở nơi cần thiết để giải thích logic.

6. Xử lý ngoại lệ khéo léo

Luôn xử lý lỗi có thể xảy ra và đưa thông báo thân thiện cho người dùng.

6 CÂU HỎI PHÒNG VẤN QUAN TRỌNG

- Python là gì và các đặc điểm của nó?
- Giải thích list và tuple khác nhau ra sao.
- Sự khác nhau giữa `==` và `is` là gì?
- `*args` và `**kwargs` là gì?
- Giải thích các khái niệm OOP kèm ví dụ.
- `self` dùng để làm gì trong Python?

7 Ý TƯỞNG DỰ ÁN NHỎ

- Ứng dụng To-Do List (CLI)
- Hệ thống quản lý sinh viên
- Sổ liên lạc (Contact Book)
- Công cụ sắp xếp file (File Organizer)
- Trình tạo mật khẩu (Password Generator)

★ Luyện tập mỗi ngày, xây dựng dự án nhỏ, đọc code người khác, giải quyết vấn đề thực tế.



PYTHON

GHI CHÚ TOÀN TẬP

Trang 11

TRANG 11 — MODULE, PACKAGE & VIRTUAL ENV

Các chủ đề này giúp ta tổ chức dự án lớn, tái sử dụng code và quản lý môi trường (environment) một cách chuyên nghiệp.

1 MODULE

- Module là một file Python (.py) chứa hàm, class hoặc biến.
- Giúp tái sử dụng code và tổ chức tốt hơn.
- Ta có thể import module để dùng code trong đó.

```
# math_utils.py
def add(a, b):
    return a + b
def sub(a, b):
    return a - b
```

```
# main.py
import math_utils
print(math_utils.add(5,3)) # 8
print(math_utils.sub(5,3)) # 2
```

Dùng `import module_name` hoặc `from module_name import function_name` ★

2 PACKAGE

- Package là tập hợp các module trong một thư mục.
- Phải có file `__init__.py` (có thể để trống).
- Giúp tổ chức các module liên quan.

Cấu trúc Package

```
my_package/
├── __init__.py
├── math_utils.py
├── string_utils.py
├── sub_package/
│   ├── __init__.py
│   └── date_utils.py
```

Sử dụng package

```
from my_package import math_utils
result = math_utils.add(10, 20)
```

3 BIẾN ĐẶC BIỆT TRONG MODULE

- Mỗi module có sẵn một số biến đặc biệt cung cấp thông tin về module đó.

```
print(__name__) # tên module hiện tại
print(__file__) # đường dẫn file
print(__doc__) # chuỗi docstring
```

Quan trọng nhất:
`if __name__ == "__main__":`
Chỉ chạy khi file được thực thi trực tiếp.

```
# example.py
def greet():
    print("Hello from function")
if __name__ == "__main__":
    print("This file is run directly")
    greet()
```

4 MÔI TRƯỜNG ẢO (VIRTUAL ENVIRONMENT)

- Môi trường ảo cô lập các dependency của dự án.
- Mỗi dự án có thể có môi trường và phiên bản riêng.

Tạo & kích hoạt

```
# Tạo
python -m venv myenv
# Windows
myenv\Scripts\activate
# macOS/Linux
source myenv/bin/activate
```

Thoát

deactivate

Vì sao dùng Venv?
✓ Cô lập dependency
✓ Dễ quản lý dự án
✓ Tránh xung đột phiên bản

5 IMPORT TỪ PACKAGE

- Ta có thể import cả module, hàm cụ thể, hoặc dùng alias.

```
# Import cả module
from my_package import math_utils
# Import hàm cụ thể
from my_package.math_utils import add
# Dùng alias
import my_package.math_utils as mu
```

Best Practice
✓ Dùng import tương minh
✓ Tránh import *
✓ Giữ cấu trúc rõ ràng

6 IMPORT TƯƠNG ĐỐI (BÊN TRONG PACKAGE)

- Dùng để import module trong cùng package. Có 2 loại.

```
# trong sub_package/date_utils.py
from .. import math_utils
# đi lên 1 cấp
result = math_utils.add(10,5)
```

```
# trong sub_package/date_utils.py
from . import helper
# import helper.py cùng package
```

Import tương đối chỉ hoạt động khi chạy module bằng `-m` từ package root.

7 QUY TRÌNH LÀM VIỆC HOÀN CHỈNH



★ Code tổ chức tốt hôm nay sẽ trở thành dự án mạnh mẽ ngày mai! 🐍



PYTHON

GHI CHÚ TOÀN TẬP

Trang 12

TRANG 12 — CẤU TRÚC DỮ LIỆU

Cấu trúc dữ liệu (data structures) giúp ta lưu trữ, tổ chức và quản lý dữ liệu hiệu quả. Python có sẵn các cấu trúc dữ liệu mạnh mẽ và dễ dùng.

1 LIST

- List là tập hợp có thứ tự, thay đổi được.
- Có thể lưu nhiều kiểu dữ liệu khác nhau.
- Định nghĩa bằng dấu ngoặc vuông [].

```
# Ví dụ
fruits = ["apple", "banana", "cherry", 10]
print(fruits[0]) # apple
fruits.append("mango") # thêm cuối
fruits.insert(1, "grapes") # chèn tại 1
fruits.remove("banana")
print(fruits)
```

Phương thức thường dùng

- append(x) → thêm
- insert(x) → chèn
- remove(x) → xóa
- pop() → xóa cuối
- sort() → sắp xếp
- reverse() → đảo ngược

2 TUPLE

- Tuple là tập hợp có thứ tự, không thay đổi được.
- Nhanh hơn và tốn ít bộ nhớ hơn list.
- Định nghĩa bằng dấu ngoặc tròn ().

```
# Ví dụ
point = (10, 20)
print(point[0]) # 10
# point[0] = 15 ❌ Không được phép
print(len(point)) # 2
```

Dùng tuple khi dữ liệu không nên thay đổi. Ví dụ: tọa độ, bản ghi CSDL, hằng số.

3 SET

- Set là tập hợp không có thứ tự, phần tử duy nhất.
- Không cho phép trùng lặp.
- Định nghĩa bằng dấu ngoặc nhọn {}.

```
# Ví dụ
s = {1, 2, 3, 3, 4, 5}
print(s) # {1, 2, 3, 4, 5}
s.add(6)
s.remove(2)
print(s) # {1, 3, 4, 5, 6}
```

Phương thức tập hợp

- a = {1, 2, 3}
- b = {3, 4, 5}
- a|b → Hợp {1, 2, 3, 4, 5}
- a&b → Giao {3}
- a-b → Hiệu {1, 2}

4 DICTIONARY

- Dictionary lưu dữ liệu theo cặp key : value.
- Key phải duy nhất và bất biến.
- Định nghĩa bằng {} với dạng key : value.

```
# Ví dụ
student = {"name": "Zaid", "age": 21, "city": "Delhi"}
print(student["name"]) # Zaid
student["age"] = 22 # cập nhật
student["course"] = "Python" # thêm key mới
del student["city"] # xóa key
```

Phương thức Dict

- keys() → mọi key
- values() → mọi value
- items() → cặp key-value
- get(key) → lấy value
- update(d) → cập nhật

5 STRING

- String là một dãy ký tự.
- Bất biến (không thể thay đổi).
- Định nghĩa bằng dấu nháy đơn hoặc kép.

```
text = "Hello Python!"
print(text[0]) # H
print(text[-1]) # !
print(len(text)) # 12
print(text.startswith("Hello")) # True
print(text.upper()) # HELLO PYTHON!
```

6 BOOLEAN

- Boolean biểu diễn True hoặc False.
- Dùng trong điều kiện và phép toán logic.

```
a = True
b = False
print(type(a)) # <class 'bool'>
print(a and b) # False
print(a or b) # True
print(not b) # True
```

Toán tử logic

- and → True nếu cả hai True
- or → True nếu có ít nhất một True
- not → đảo ngược giá trị
- in / not in → kiểm tra thành viên

7 BẢNG SO SÁNH CẤU TRÚC DỮ LIỆU

Cấu trúc	Thứ tự	Thay đổi	Trùng lặp	Chỉ số/Key
List	Yes	Yes	Yes	Index (0,1,2,...)
Tuple	Yes	No	Yes	Index (0,1,2,...)
Set	No	Yes	No	Không có chỉ số
Dictionary	Yes	Yes	No	Key (duy nhất)
String	Yes	No	Yes	Index (0,1,2,...)

KHI NÀO DÙNG CÁI GÌ?

- ✓ Dùng **List** → khi thứ tự và khả năng thay đổi quan trọng
- ✓ Dùng **Tuple** → khi dữ liệu cố định và an toàn
- ✓ Dùng **Set** → khi tính duy nhất là quan trọng
- ✓ Dùng **Dictionary** → khi cần ánh xạ key-value
- ✓ Dùng **String** → cho thao tác văn bản và ký tự

8 THỰC HÀNH NHANH

1. List: Xóa hết số chẵn khỏi list.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]
```

2. Tuple: Đếm số lần 'a' xuất hiện.

```
t = ('a', 'b', 'a', 'c', 'a')
```

3. Set: Tìm hợp và giao.

```
x = {1, 2, 3}
```

4. Dict: Hoán đổi key và value.

```
d = {'a': 1, 'b': 2}
```

5. String: Đảo ngược chuỗi mà không dùng reverse().



PYTHON

GHI CHÚ TOÀN TẬP

Trang 13

TRANG 13 — QUẢN LÝ PACKAGE, VENV & PIP

Package giúp Python mạnh mẽ hơn. Hãy cùng tìm hiểu cách quản lý chúng bằng pip và làm việc trong môi trường sạch bằng virtual environment (venv).

1 PACKAGE LÀ GÌ?

- Package là tập hợp các module và file Python liên quan.
- Bổ sung thêm chức năng cho dự án.
- Ta dùng package từ Python Package Index (PyPI).

Dự án Python của bạn

Packages (libraries)
• requests
• numpy
• pandas
• flask
• v.v...

2 PIP LÀ GÌ?

- pip là trình cài đặt package của Python.
- Có sẵn cùng Python (3.4+).
- Dùng để cài, gỡ và quản lý package từ PyPI.

```
pip install package_name  
pip list  
pip show package_name  
pip uninstall package_name
```

pip là cách dễ nhất để lấy package trong Python! ✨

3 KIỂM TRA PHIÊN BẢN PYTHON & PIP

- Dùng các lệnh này để kiểm tra Python và pip đã được cài chưa.

```
python --version  
pip --version
```

Ví dụ kết quả:

```
python 3.12.3  
pip 24.0
```

Giữ Python và pip cập nhật để có độ tin cậy tối hơn! ✨

4 LỆNH PIP CƠ BẢN

Lệnh	Mô tả
pip install <pkg>	Cài package
pip uninstall <pkg>	Gỡ package
pip list	Liệt kê package đã cài
pip show <pkg>	Xem chi tiết package
pip freeze	Liệt kê (cho requirements.txt)
pip search <kw>	Tìm package

5 CÀI ĐẶT PACKAGE

- Dùng pip install để cài package bất kỳ từ PyPI.

```
pip install requests
```

Ví dụ kết quả:

```
Collecting requests  
Downloading requests-2.31.0...  
Successfully installed requests-2.31.0
```

Thao tác này tải và cài package bạn yêu cầu từ PyPI. ✨

6 SỬ DỤNG PACKAGE ĐÃ CÀI

- Import package và dùng tính năng của nó trong code.

```
import requests
```

```
response = requests.get(  
    "https://api.github.com/"  
)  
print(response.status_code)  
print(response.json())
```

Giữ bạn có thể dùng package trong chương trình! ✨

7 GỠ CÀI ĐẶT PACKAGE

```
pip uninstall requests
```

```
Found existing installation: requests 2.31.0  
Uninstalling requests-2.31.0:  
Successfully uninstalled requests-2.31.0
```

Thao tác này gỡ package khỏi hệ thống của bạn. ✨

8 LIỆT KÊ PACKAGE ĐÃ CÀI

```
pip list
```

Package	Version
requests	2.31.0
numpy	1.26.4
pandas	2.2.0

Hiển thị mọi package đã cài trong môi trường hiện tại. ✨

9 XEM CHI TIẾT PACKAGE

```
pip show requests
```

```
Name: requests  
Version: 2.31.0  
Summary: Python HTTP for Humans.  
Author: Kenneth Reitz  
License: Apache 2.0
```

Hữu ích để xem chi tiết như phiên bản, vị trí, license... ✨

10 FILE REQUIREMENTS

- Dùng pip freeze để lưu các package đã cài. Giúp chia sẻ đúng phiên bản với người khác.

```
pip freeze > requirements.txt
```

requirements.txt (ví dụ)

```
requests==2.31.0  
numpy==1.26.4  
pandas==2.2.0  
flask==3.0.0
```

```
pip install -r requirements.txt
```

11 MÔI TRƯỜNG ẢO (VENV)

- Virtual environment giữ dự án của bạn tách biệt. Mỗi dự án có thể có dependency riêng.

```
python -m venv myenv
```

Các bước kích hoạt:

```
# Windows  
myenv\Scripts\activate  
# macOS/Linux  
source myenv/bin/activate
```

Thoát:

```
deactivate
```

12 TẠI SAO DÙNG VENV?

- ✓ Giữ dependency của dự án tách riêng
- ✓ Tránh xung đột giữa các dự án khác nhau
- ✓ Dễ quản lý và thiết lập gọn gàng
- ✓ Cần thiết cho production và chia sẻ

Sạch sẽ hơn. Tổ chức tốt hơn. Developer hạnh phúc hơn! ✨

13 QUY TRÌNH LÀM VIỆC DỰ ÁN PYTHON

- Tạo môi trường ảo → `python -m venv myenv`
- Kích hoạt → `source myenv/bin/activate`
- Cài package cần thiết → `pip install package_name`

14 LỖI THƯỜNG GẶP & CÁCH SỬA

Lỗi	Cách sửa
'pip' not recognized	Kiểm tra Python trong PATH
ModuleNotFoundError	Cài package bằng pip
Permission denied	Dùng --user hoặc sudo (Linux/macOS)

15 GHI NHỚ CHÍNH

- ✓ Package giúp Python mạnh mẽ hơn
- ✓ pip là trình quản lý package
- ✓ Dùng venv để cô lập dự án
- ✓ Dùng pip freeze cho requirements



PYTHON

GHI CHÚ TOÀN TẬP

Trang 14

TRANG 14 — COMPREHENSION, DECORATOR & CONTEXT MANAGER

Các tính năng Python nâng cao giúp code ngắn gọn, hiệu quả và Pythonic hơn.

1 LIST COMPREHENSION

- Cách ngắn gọn để tạo list.
- Dễ đọc và thường nhanh hơn vòng lặp.

```
# Cú pháp
new_list = [expression for item in iterable
            if condition]

# Ví dụ
squares = [x*x for x in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]
even_nums = [x for x in range(10) if x%2==0]
print(even_nums) # [0, 2, 4, 6, 8]
```

Lợi ích

✓ Code ngắn gọn ✓ Dễ đọc hơn ✓ Ít lỗi hơn ✓ Thường nhanh hơn

2 DICTIONARY COMPREHENSION

- Tạo dictionary chỉ trong một dòng.
- Cập key-value được sinh động.

```
# Cú pháp
new_dict = {key_expr: value_expr for item in iterable
            if condition}

# Ví dụ
squares = {x: x*x for x in range(1, 6)}
print(squares) # {1:1,2:4,3:9,4:16,5:25}

char_count = {ch: word.count(ch) for ch in set(word)}
# với "hello" → {'o':1, 'h':1, 'e':1, 'l':2}
```

Dùng comprehension khi logic đơn giản và dễ đọc.

3 SET COMPREHENSION

- Tạo set bằng dấu ngoặc nhọn.
- Chỉ lưu các giá trị duy nhất.

```
# Cú pháp
new_set = {expression for item in iterable
           if condition}

# Ví dụ
squares = {x*x for x in range(1, 6)}
print(squares) # {1, 4, 9, 16, 25}
vowels = {ch for ch in "programming" if ch in 'aeiou'}
print(vowels) # {'a', 'o', 'i'}
```

Set comprehension tự động loại bỏ trùng lặp. ✓

4 BIỂU THỨC GENERATOR

- Giống list comprehension nhưng dùng () thay vì [].
- Sinh giá trị theo yêu cầu (lazy evaluation).

```
# Cú pháp
(expression for item in iterable if condition)

# Ví dụ
gen = (x*x for x in range(1, 6))
print(next(gen)) # 1
print(next(gen)) # 4
print(list(gen)) # [9, 16, 25]
```

Vì sao dùng? ✓ Tiết kiệm bộ nhớ ✓ Tốt cho dữ liệu lớn ✓ Dùng trong sum(), max(), any()...

5 DECORATOR

- Decorator thay đổi hành vi của hàm.
- Cài đặt bằng ký hiệu @.

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello, Python!")

say_hello()
```

Trường hợp dùng phổ biến: Logging, Access control, Timing, Validation, Caching...

6 NHIỀU DECORATOR

- Decorator được áp dụng từ dưới lên trên.
- Decorator gần nhất được áp dụng trước.

```
def decorator_one(func):
    def wrapper():
        print("Decorator One")
        func()
    return wrapper

def decorator_two(func):
    def wrapper():
        print("Decorator Two")
        func()
    return wrapper

@decorator_one
@decorator_two
def greet():
    print("Hello!")

greet()
# Output: Decorator One / Decorator Two / Hello!
```

7 CONTEXT MANAGER

- Quản lý tài nguyên (như file) tự động.
- Đảm bảo setup và cleanup dùng câu lệnh with.

```
# context manager dùng sẵn
with open("data.txt", "r") as f:
    data = f.read()
    print(data)

# context manager tự viết
class MyManager:
    def __enter__(self):
        print("Entering context")
        return self
    def __exit__(self, *a):
        print("Exiting context")
    with MyManager() as m:
        print("Inside context")
```

Vì sao dùng with? ✓ Tự động tài nguyên ✓ Code sạch hơn ✓ Xử lý ngoại lệ chuẩn

8 MODULE ITERTOOLS

- Cung cấp công cụ lập nhanh, tiết kiệm bộ nhớ.

Hàm	Mô tả	Ví dụ
count(start)	Bộ đếm vô hạn	count(10)-10,11,12...
cycle(iterable)	Lặp lại phần tử	cycle([1,2])-1,2,1,2...
repeat(elem,n)	Lặp phần tử n lần	repeat(5,3)-5,5,5
chain(a,b)	Nối các iterable	chain('ab','cd')-a b c d
islice(it,start,stop)	Cắt lát iterator	islice(count(),1,4)-1,2,3
product(a,b)	Tích Descartes	product([1,2], 'ab')-(1,a)...

Dùng itertools khi làm việc với dữ liệu lớn hoặc vô hạn. Tiết kiệm bộ nhớ và tăng hiệu năng.

9 Ý TƯỞNG DỰ ÁN NHỎ

TRÌNH PHÂN TÍCH FILE LOG (LOG FILE ANALYZER)

Đọc file log, đếm số lỗi bằng generator expression, và dùng decorator để đo thời gian xử lý.

10 CÂU HỎI PHÒNG VẤN

- 1. Khác biệt giữa list và generator expression là gì?



PYTHON

GHI CHÚ TOÀN TẬP

Trang 15

TRANG 15 — BEST PRACTICES, FILE I/O, MODULE & THÊM

Viết code Python sạch, mạnh mẽ và dễ bảo trì như một lập trình viên chuyên nghiệp.

1 CÁC NGUYÊN TẮC TỐT

- ✓ Tuân theo chuẩn PEP 8
- ✓ Dùng tên rõ nghĩa và nhất quán
- ✓ Viết hàm nhỏ, tập trung
- ✓ Thêm comment và docstring
- ✓ Xử lý ngoại lệ khéo léo
- ✓ Tránh lặp code (nguyên tắc DRY)
- ✓ Dùng list/set/dict comprehension
- ✓ Giữ code đơn giản (nguyên tắc KISS)

Ví dụ - Tốt vs Chưa tốt

```
Chưa tốt:
def sum(a,b): return(a+b)

Tốt:
def add(a:int,b:int)->int:
    """Trả về tổng 2 số."""
    return a+b
```

2 XỬ LÝ LỖI

- Dùng try, except, else, finally.
- Giúp chương trình không bị crash.
- Luôn xử lý ngoại lệ cụ thể.

```
try:
    num = int(input("Enter number: "))
    result = 10 / num
except ValueError:
    print("Please enter a valid integer.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print("Result:", result)
finally:
    print("Execution completed.")
```

💡 Vì sao dùng finally? Nó luôn chạy dù có chuyện gì xảy ra (kể cả khi có exception).

3 TYPE HINTING (TÙY CHỌN)

- Type hint giúp code dễ đọc và bắt lỗi sớm.
- Giới thiệu từ Python 3.5+, hỗ trợ đầy đủ từ 3.10+.

```
def greet(name: str, age: int) -> str:
    return f"Hello (name), you are {age} years old."

numbers: list[int] = [1, 2, 3]
person: dict[str, str] = {"name": "Zaid", "city": "Delhi"}
```

Lợi ích ✓ Code dễ đọc hơn ✓ Phát hiện lỗi sớm ✓ IDE hỗ trợ tốt hơn

4 LÀM VIỆC VỚI FILE (I/O)

- Dùng để đọc và ghi file.
- Luôn đóng file sau khi dùng (dùng with open để an toàn).

Đọc file

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

Ghi file

```
with open("data.txt", "w") as f:
    f.write("Hello Python!")
```

Nối thêm file

```
with open("data.txt", "a") as f:
    f.write("\nNew line added.")
```

🔑 Mode khác: r+, w+, a+, rb, wb, ab cho nh phân và cập nhật.

5 MODULE VS PACKAGE

Module

- Một file Python (.py)
- Chứa hàm, class hoặc biến
- Ví dụ:

```
import math_utils
math_utils.add(2,3)
```

Package

- Thư mục chứa nhiều module
- Phải có file __init__.py
- Import thông dụng:

```
import module
from module import func
```

6 CÁC HÀM DỰNG SẴN HỮU ÍCH

len(x) → độ dài đối tượng **abs(x)** → giá trị tuyệt đối
type(x) → kiểu của đối tượng **round(x,n)** → làm tròn n chữ số
id(x) → địa chỉ bộ nhớ **sorted(iter)** → trả về list đã sắp xếp
int(x) → chuyển thành số nguyên **enumerate(iter)** → chỉ số + giá trị
str(x) → chuyển thành chuỗi **zip(a,b)** → kết hợp iterable
sum(iter) → tổng các phần tử **map(func, iter)** → áp dụng hàm
max(iter) → giá trị lớn nhất **filter(func, iter)** → lọc phần tử
min(iter) → giá trị nhỏ nhất

7 CÁC MẪU PYTHON PHỔ BIẾN

Hoán đổi 2 số

```
a, b = b, a # cách Pythonic
```

List Comprehension

```
squares = [x*x for x in range(6)]
print(squares) # [0,1,4,9,16,25]
```

Kiểm tra key tồn tại

```
if "name" in person:
    print(person["name"])
```

Dictionary Comprehension

```
squares = {x: x*x for x in range(5)}
print(squares) # {0:0,1:1,2:4,3:9,4:16}
```

8 VÍ DỤ DOCSTRING

```
def divide(a: int, b: int) -> float:
    """Chia a cho b và trả về kết quả.

    Args:
        a (int): Số bị chia
        b (int): Số chia

    Returns:
        float: Kết quả phép chia

    Raises:
        ZeroDivisionError: Nếu b bằng 0

    """
    if b == 0:
        raise ZeroDivisionError("b không được bằng 0")
    return a / b
```

Truy cập bằng help() >>> help(divide) Hiển thị docstring của hàm trong module __main__

9 CẨM NANG THU NHỎ PYTHON

Nhóm	Cú pháp / Ví dụ	Ý nghĩa
Print	print("Hello")	Xuất văn bản
Input	input("Name: ")	Nhận dữ liệu người dùng
If-Else	if cond: ... else: ...	Câu lệnh điều kiện
For Loop	for i in range(n): ...	Lặp n lần
While Loop	while cond: ...	Lặp khi điều kiện còn True

XEM NHANH TOÁN TỬ

Số học: + - * / // % **

So sánh: == != > < >= <=

Logic: and or not

10 LỜI KẾT

- ✓ Python đơn giản, mạnh mẽ và đa năng.
- ✓ Luyện tập đều đặn và xây dựng dự án.
- ✓ Đọc tài liệu để hiểu sâu hơn.
- ✓ Viết code sạch và học từ những lỗi sai.

🙌 Cảm ơn bạn đã theo dõi trọn bộ Python Complete Notes! 🐍